

UNIVERSITÀ DEGLI STUDI DI
MILANO-BICOCCA

ADVANCED MACHINE LEARNING
PROGETTO FINALE

Creazione ed analisi di due classificatori di immagini di
animali acquatici

Autori:

Marco Ferioli - 879277 - m.ferioli@campus.unimib.it
Francesco Trolli - 889039 - f.trolli1@campus.unimib.it

Appello gennaio 2026



Sommario

Questo progetto affronta un problema di classificazione in 23 classi su un dataset Kaggle contenente circa 13700 immagini di animali marini. Dopo una fase di pulizia dei dati, che include la rimozione delle immagini duplicate e quelle con dimensioni non coerenti, è stata progettata una pipeline di transfer learning e fine tuning basata su due CNN, **ResNet50** ed **EfficientNetB3**. La strategia di training integra in modo sistematico: data augmentation, pesi di classe per gestire lo sbilanciamento, learning rate adattivo, early stopping e dropout, permettendo il confronto in maniera coerente delle diverse configurazioni dei modelli. **ResNet50** raggiunge sul test set un macro F1-score di 0,715, mentre **EfficientNetB3** ottiene valori significativamente superiori con 0,865, confermando la superiorità di quest'ultima architettura. Questi risultati dimostrano come la pulizia accurata del dataset, la progettazione di una pipeline di training ben strutturata e l'uso di architetture CNN moderne come **EfficientNet** siano elementi fondamentali per ottenere prestazioni elevate nella classificazione multi-classe di specie marine.

1 Introduzione

L'obiettivo del progetto è stato quello di creare una pipeline di addestramento per un modello di machine learning con lo scopo di risolvere un problema di classificazione multipla di un dataset di immagini. Per mettere in atto tale obiettivo, sono stati sviluppati, utilizzando l'ambiente di lavoro "Google Colab", due notebook in python che implementano le procedure preliminari di analisi e modifica del dataset, seguite dalla fase di addestramento, validazione e test del modello. In particolare, ciascun notebook prende in considerazione una specifica architettura di CNN; in questo caso l'obiettivo era quello di mettere a confronto due modelli architetturelmente differenti con lo scopo di valutarne le prestazioni e le caratteristiche. Sono state utilizzate tecniche di transfer learning e di fine tuning per la fase di addestramento unite a tecniche di regolarizzazione, come dropout ed early stopping, per evitare overfitting; inoltre sono state applicate anche procedure di learning rate adattivo e di data augmentation.

2 Analisi del dataset

Il dataset di riferimento è stato identificato nella challenge di Kaggle indicata nel seguente link . In particolare, il dataset contiene un insieme di circa 13700 immagini di specie di animali marini vertebrati e invertebrati suddivise in 23 classi differenti; le immagini della stessa specie sono collocate nella stessa directory. Dopo aver caricato il dataset all'interno dello spazio di lavoro, sono state effettuate due operazioni:

- **Identificazione di immagini duplicate.** Poichè alcuni utenti segnalavano la presenza di immagini duplicate tra le directory del dataset, è stato scritto un semplice script per identificarle. Quest'ultimo si basa sulla memorizzazione degli hash MD5 dei vari file in un dizionario chiave-valore; poichè due immagini uguali hanno lo stesso hash, è possibile determinare velocemente i duplicati. Questa operazione si è rivelata molto importante, perché ha permesso di identificare circa 66 immagini duplicate tra cartelle differenti; ad esempio l'immagine di una balena compariva anche all'interno della directory associata alle aragoste. Dato il numero non proibitivo di duplicati, questi sono stati rintracciati manualmente nelle cartelle e sono stati eliminati quelli associati alla classe errata.
- **Analisi sulla dimensione dell'immagine.** All'interno della descrizione del dataset preso in considerazione vi è indicato che le immagini hanno dimensione $300 \times N$ o $N \times 300$, dove N rappresenta un numero intero minore di 300 che varia in base all'immagine. È stato predisposto uno script per verificare se ci fossero delle immagini con una dimensione che non rispettasse quella descritta inizialmente; in particolare sono state trovate circa 30 immagini, che sono state eliminate. La motivazione di tale scelta è legata alla tecnica di resize utilizzata per avere a disposizione immagini della stessa dimensione; effettuando tale operazione su immagini troppo grosse, si sarebbe potuta verificare la situazione in cui venivano perse la forma e proporzione dell'animale rappresentato. Dato il numero basso di immagini trovate, la loro rimozione non rappresenta una grande perdita a livello di dati a disposizione.

Durante lo svolgimento di tali operazioni, è stata condotta anche una semplice analisi esplorativa visiva del contenuto delle directory. Tale operazione ha evidenziato un ulteriore problema: all'interno del dataset sono presenti

alcune immagini mal classificate. Consideriamo ad esempio la Figura 1; come è possibile notare l'immagine mostra tre specie differenti: una balena, uno squalo balena ed un'orca. Il problema principale è che tutte queste immagini sono classificate come "balena", perché sono presenti all'interno della stessa directory. Questa situazione può portare i modelli addestrati successivamente ad imparare features scorrette producendo delle falle durante la sezione di addestramento e generando quindi delle prestazioni non entusiasmanti. Ad esempio, osservando molteplici immagini di squali balena, il modello potrebbe imparare scorrettamente che le normali balene hanno una pinna dorsale, la quale rappresenta invece una caratteristica distintiva di altre specie. Il problema è altresì amplificato dal fatto che tali specie compaiono in directory differenti; ad esempio ci sono immagini di squali balena sia all'interno della directory "squalo" che all'interno della directory "balena".



Figura 1: Specie differenti registrate all'interno della stessa directory

Successivamente, è stata condotta un'indagine sul numero di immagini presenti nel dataset associate a ciascuna classe. L'obiettivo di tale operazione è stato quello di valutare il bilanciamento del dataset. La Figura 2 mette chiaramente in evidenza che il dataset è sbilanciato rispetto alla classe "Turtle Tortoise"; è importante gestire tale problema per evitare che i modelli imparino a prevedere sempre la classe maggioritaria.

L'ultima operazione effettuata sul dataset riguarda la divisione dei dati nelle porzioni di training, validation e test. In particolare, il criterio di split se-

lezionato definisce che l'80% dei dati viene assegnato al dataset di training, mentre le porzioni di validation e test dividono equamente il restante 20%. Durante lo svolgimento di tale operazione viene altresì definita la suddivisione interna in batch e le immagini vengono ridimensionate per renderle coerenti con il livello di input del modello.

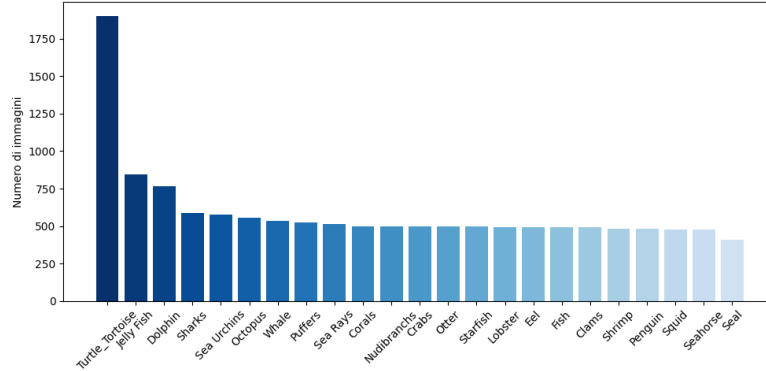


Figura 2: Distribuzione delle immagini per classe

3 Approccio metodologico applicato

3.1 Descrizione della pipeline di lavoro

Dato che il problema trattato è relativo alla classificazione di immagini, entrambi i modelli presi in considerazione sono delle Convolutional Neural Network (CNN). Considerando la dimensione del dataset di training pari a circa undicimila immagini, è stata fin da subito scartata l'idea di definire ed addestrare una CNN da zero; per questa ragione sono state prese in considerazione strategie di transfer learning e fine tuning. Per entrambi i modelli è stata predisposta la seguente pipeline:

1. **Preparazione del modello e pre-processing dell'input.** In questa fase il modello viene caricato in memoria e gli vengono assegnati i pesi relativi all'addestramento sul dataset "ImageNet"; successivamente viene effettuato un preprocessing dei dati di training in modo tale che possano essere forniti in input alla rete. Per mettere in atto tale operazione, sono stati eseguiti i comandi di `preprocess_input` messi a disposizione dalle librerie stesse.

2. **Inserimento del layer di data augmentation.** Subito dopo il livello di input del modello caricato è stato inserito un layer che effettua la procedura di data augmentation dell'immagine; le operazioni predisposte riguardano il flip orizzontale e verticale, la rotazione, la traslazione, lo zoom, la modifica del contrasto e della luminosità dell'immagine in input. Tale operazione si è rivelata fondamentale per gestire la dimensione relativamente limitata del dataset di addestramento ed ha permesso inoltre di "irrobustire" il modello nel riconoscere immagini più complesse. Si è rivelato altresì importante definire e riconoscere dei limiti nelle operazioni di data augmentation utilizzate; ad esempio non è stato possibile predisporre una variazione eccessivamente alta nella luminosità dell'immagine, perché le foto scattate sott'acqua avrebbero perso completamente il loro contenuto visivo.
3. **Sostituzione del classificatore ed addestramento con transfer learning.** Il classificatore originale del modello è stato sostituito con un nuovo classificatore inizializzato con pesi casuali. Successivamente, la restante porzione della rete è stata "congelata" ed è stato avviato l'addestramento del nuovo classificatore; in questo caso è stata messa in atto una vera e propria operazione di transfer learning. Infatti è stata sfruttata la capacità della rete di produrre features a partire dalle immagini in input per allenare il classificatore nel risolvere il problema.
4. **Fine tuning della rete.** In questo caso sono stati scongelati un certo numero di layer convoluzionali dei modelli ed è stato diminuito il learning rate; l'obiettivo è stato quello di continuare l'addestramento della rete facendo in modo che quest'ultima si specializzasse nel riconoscere le features delle immagini fornite in input. L'obiettivo di questa fase è stato quello di verificare se la procedura di fine tuning producesse delle prestazioni migliori dell'addestramento tramite transfer learning sul validation set in modo tale da valutarne l'effettiva utilità.
5. **Valutazione sul test set.** Dopo aver terminato l'operazione di fine tuning è stata effettuata una scelta con l'obiettivo di determinare la migliore metodologia di addestramento per il problema preso in considerazione. In particolare sono stati messi a confronto la velocità di addestramento e l'accuratezza ottenuta sul validation set. Infine sono state valutate le prestazioni del modello sul test set, stampando un classification report ed una confusion matrix.

In combinazione con tale pipeline sono state adottate strategie di early stopping e learning rate adattivo. In entrambi i casi viene monitorata la loss in fase di validazione; in particolare si possono verificare due situazioni differenti. Il primo caso è quello in cui la funzione di loss ha un tipico andamento "altalenante". Questo comportamento è un chiaro segnale che il learning rate è troppo alto; di conseguenza viene diminuito. Il secondo caso è quello in cui la funzione di loss ha un andamento crescente; questa situazione segnala che il modello sta entrando in una situazione di overfitting e la soluzione è quella di interrompere l'addestramento e ripristinare il modello all'istante (epoca) in cui produceva l'accuratezza massima in fase di validazione. Un'ulteriore tecnica utilizzata per diminuire l'overfitting del modello è quella del dropout, che consiste nel disattivare casualmente dei neuroni in fase di addestramento; questa tecnica è stata utilizzata durante l'addestramento del nuovo classificatore nelle fasi di transfer learning e fine tuning.

3.2 Descrizione del modello ResNet

Il primo modello considerato è una **ResNet50**, una rete convoluzionale composta da 50 strati. A differenza delle reti tradizionali, che tendevano a peggiorare le proprie prestazioni all'aumentare degli strati, a causa della difficoltà nel propagare correttamente il gradiente durante la fase di backpropagation, **ResNet** risolve questo problema introducendo le connessioni residue (skip connections). Tali connessioni permettono di risolvere il problema del gradiente evanescente, consentendo al segnale di flusso di "saltare" uno o più livelli. L'architettura (Figura 3) impiega operazioni di Batch Normalization e funzioni di attivazione ReLU, seguite da livelli di pooling e da un classificatore fully-connected finale. Tali caratteristiche rendono la **ResNet50** adatta al riconoscimento di immagini complesse, mantenendo comunque tempi di addestramento accettabili per dataset di grandi dimensioni.

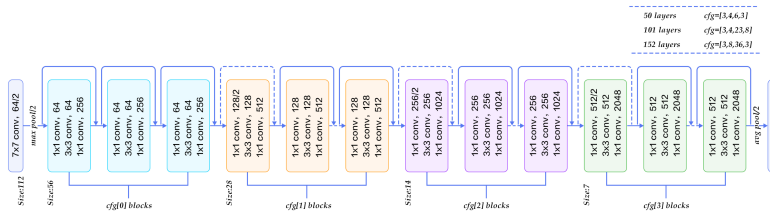


Figura 3: Architettura ResNet50

3.3 Descrizione del modello di EfficientNet

Il secondo modello addestrato tramite le metodologie esposte in precedenza è una rete di tipo **EfficientNetB3**; si tratta di una famiglia di CNN (Figura 4), che ha l'obiettivo di produrre performance elevate tramite l'utilizzo di un numero limitato di risorse computazionali. Prima di **EfficientNet**, i modelli di machine learning, destinati alla gestione di immagini, cercavano di migliorare l'accuratezza aggiungendo più layers o aumentando la dimensione di questi ultimi. Questa metodologia di miglioramento ha reso i modelli tradizionali molto pesanti e computazionalmente onerosi; l'accuratezza guadagnata diventava marginale rispetto alla potenza di calcolo richiesta.

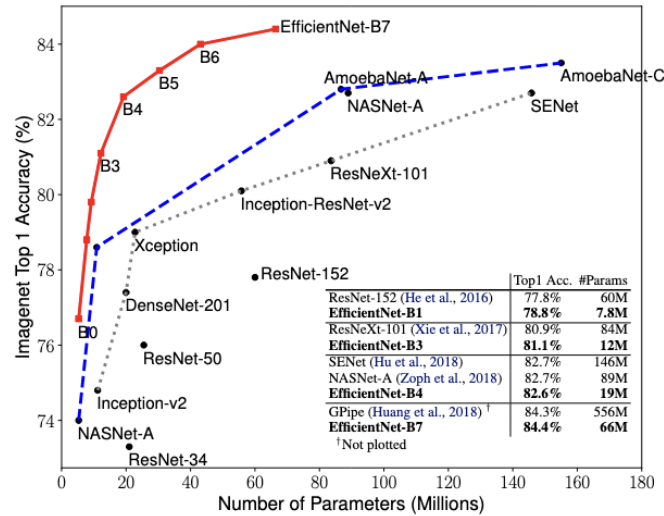


Figura 4: Differenze tra i modelli di EfficientNet per numero di parametri [1]

EfficientNet adotta un approccio differente; l'idea fondamentale è quella di aumentare l'ampiezza e profondità della rete, insieme alla risoluzione dell'immagine, in modo bilanciato. Questo approccio è denominato "compound scaling" (Figura 5) ed ha permesso di creare modelli piccoli e veloci con la capacità di raggiungere prestazioni migliori rispetto a reti come **ResNet** e **DenseNet**. Da un punto di vista pratico, è presente un parametro ϕ che regola il compound scaling; ciò permette di definire otto modelli differenti di **EfficientNet**. Nel caso del progetto realizzato, è stato preso in considerazione il modello **EfficientNetB3**, che ha garantito un compromesso tra l'accuratezza ottenuta ed il tempo necessario per l'addestramento.

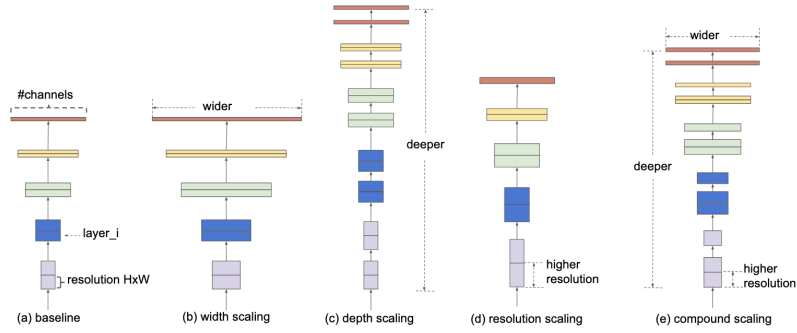


Figura 5: Rappresentazione grafica del "compound scaling" [1]

4 Risultati ottenuti

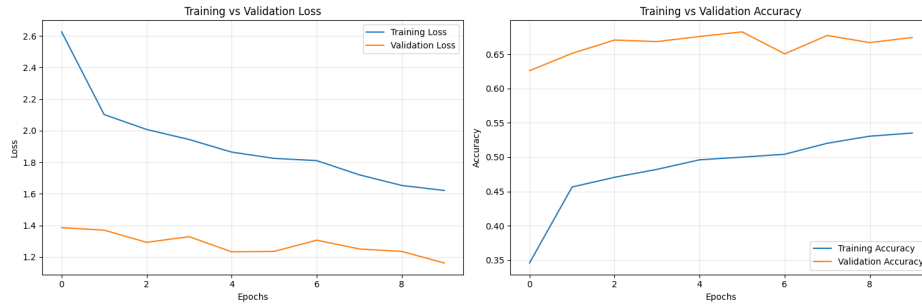


Figura 6: Curve di training e validation per TL su ResNet50



Figura 7: Curve di training e validation per FT su ResNet50

Tabella 1: Classification Report per ResNet50

Class	Precision	Recall	F1-score	Support
Macro Avg	0.740	0.715	0.715	1375
Weighted Avg	0.766	0.753	0.749	1375

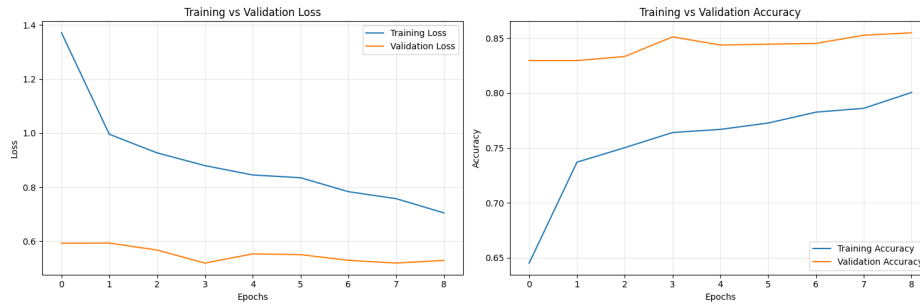


Figura 8: Curve di training e validation per TL su EfficientNetB3

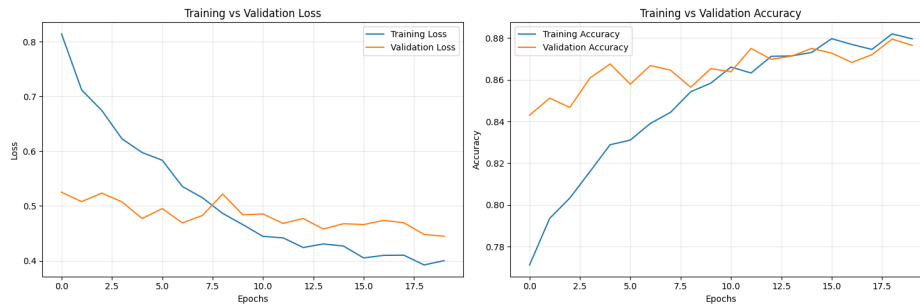


Figura 9: Curve di training e validation per FT su EfficientNetB3

Tabella 2: Classification report per EfficientNetB3

Class	Precision	Recall	F1-score	Support
Macro Avg	0.870	0.867	0.865	1375
Weighted Avg	0.888	0.885	0.884	1375

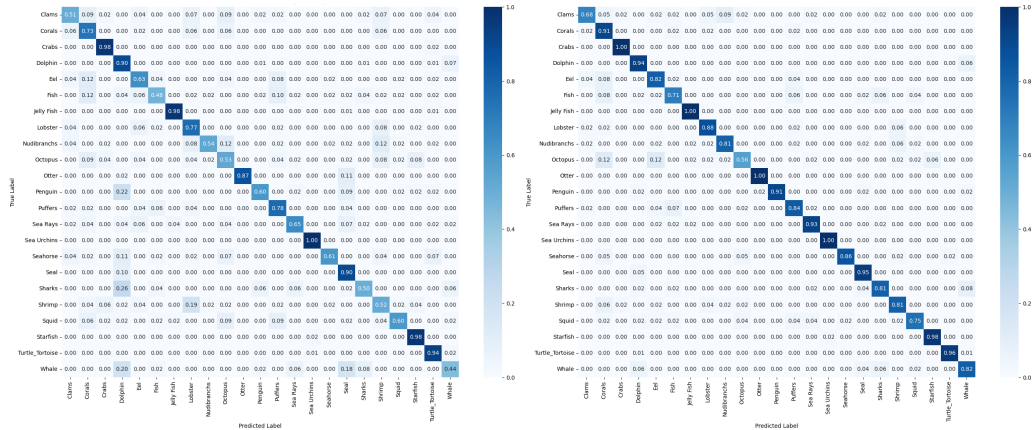


Figura 10: Matrice di confusione per ResNet50 e per EfficientNetB3

5 Discussione

Sulla base dei risultati ottenuti e rappresentati nella sezione precedente è possibile effettuare alcune considerazioni:

- I modelli costruiti non raggiungono mai una situazione di underfitting o di overfitting. Infatti, nelle immagini rappresentate in Figura 6, Figura 7, Figura 8 e Figura 9, la curva di loss associata alla fase di validazione tende a decrescere insieme alla curva di loss associata alla fase di training; allo stesso modo l'accuratezza ottenuta durante le varie epoche tende ad aumentare. Questo comportamento è più visibile durante la fase di fine tuning, rispetto alla fase di transfer learning; le tecniche di early stopping, unite al dropout, hanno permesso di controllare il fenomeno di overfitting. La curva di training, in particolare, subisce un effetto singolare, perché si trova sempre al di sotto della curva di validazione; questo comportamento non rappresenta un fenomeno di underfitting, ma è causato semplicemente dal dropout e della data augmentation, che vengono utilizzati solo durante la fase di training.
- La procedura di fine tuning produce risultati sempre migliori rispetto a quella di transfer learning per il problema preso in considerazione. Questo comportamento è evidente e può essere riscontrato analizzando i livelli di accuratezza raggiunti dalle curve di validazione rappresentate in Figura 6, Figura 7, Figura 8 e Figura 9. Ciò si verifica soprattutto

nella **ResNet50**; come è possibile notare si ottiene un miglioramento di circa il 7% di accuratezza in fase di validazione al termine dell'addestramento. È altresì possibile notare che la tecnica di early stopping subentra più velocemente in fase di transfer learning; probabilmente questa situazione accade a causa del numero minore di parametri da addestrare. Di conseguenza, i modelli valutati sui dati di test sono quelli addestrati tramite l'operazione di fine tuning.

- La rete **EfficientNetB3** raggiunge delle prestazioni significativamente superiori rispetto alla rete **ResNet50**; questo comportamento è coerente con quanto osservato dal punto di vista teorico (Figura 4). I dati della Tabella 1, messa a confronto con Tabella 2, mettono in grande evidenza tale aspetto. Per quanto riguarda la rete **EfficientNetB3**, in questo caso è stato rilevato un f1-score pari a circa 0.865 con la media aritmetica e 0.884 con la media pesata per classe. Questo comportamento suggerisce che la distribuzione sbilanciata delle classi non influisce pesantemente sulle prestazioni. Inoltre, i livelli di precision e recall sono molto simili ai valori di f1-score; ciò è indice di un relativo bilanciamento tra la capacità del modello di non assegnare etichette errate (pochi falsi positivi) e di rilevare un buon numero di istanze per ciascuna classe (pochi falsi negativi). Il comportamento della rete **ResNet50** è molto simile a quello della rete **EfficientNetB3**; la differenza sostanziale è identificabile nelle prestazioni ottenute; infatti la rete **ResNet50** raggiunge livelli di f1-score pari a 0.715 con il calcolo tramite media aritmetica e 0.749 con il calcolo tramite media pesata. Ciò indica che il modello soffre maggiormente lo sbilanciamento del dataset rispetto alla rete **EfficientNetB3**.
- Gli aspetti indicati nel punto precedente sono riscontrabili anche nelle confusion matrices, rappresentate in Figura 10. Risulta evidente che la rete **ResNet50** abbia difficoltà a rilevare tutti i campioni che appartengono ad una determinata classe (ci sono molti falsi negativi per alcune classi); ad esempio questo comportamento è riscontrabile per le classi "Fish", "Clams", "Sharks", "Shrimps". Inoltre, risulta che una delle principali classi con cui il modello genera molti falsi positivi sia il delfino, perché ci sono numerosi campioni appartenenti ad altre classi che vengono classificati come tali. Tale comportamento viene invece notevolmente mitigato per quanto riguarda la rete **EfficientNetB3**,

come possiamo notare dalla confusion matrix prodotta per tale modello. Le confusion matrices forniscono ulteriori informazioni di interesse; ad esempio le istanze delle classi "Crabs", "Jellyfish", "Otters" e "Sea Urchins" vengono classificate tutte correttamente dalla rete **EfficientNetB3** ed anche la rete **ResNet50** fornisce prestazioni ottime. La motivazione di tale comportamento è probabilmente da identificare nella struttura di tali immagini; ad esempio i granchi sono l'unico animale le cui fotografie sono state scattate sulla sabbia. Ciò rappresenta una feature molto forte per il modello e quindi per quest'ultimo è facile effettuare la classificazione.

6 Conclusioni

I risultati ottenuti confermano la validità della pipeline di addestramento proposta, dimostrando che l'integrazione di tecniche come early stopping e dropout ha prevenuto efficacemente fenomeni di overfitting e underfitting in entrambi i modelli. L'analisi comparativa evidenzia in modo netto la superiorità della procedura di fine tuning, rispetto al semplice transfer learning, risultando determinante soprattutto per la rete **ResNet50**, che ha registrato un incremento dell'accuratezza di validazione di circa il 7% al termine dell'addestramento. Il confronto architetturale mostra come **EfficientNetB3** sia il modello più performante, con un F1-score macro di 0.865 contro lo 0.715 di **ResNet50**. Tale divario è attribuibile alla maggiore robustezza di **EfficientNetB3** rispetto allo sbilanciamento delle classi e alla sua capacità di ridurre i falsi positivi, particolarmente critici in **ResNet50**. In conclusione, questo progetto evidenzia come l'impiego di architetture CNN ad alta efficienza, abbinate a una strategia di fine tuning mirata, sia determinante per affrontare dataset marini affetti da sbilanciamento e rumore. Possibili sviluppi futuri potranno concentrarsi sulla pulizia del dataset dalle etichette rumorose e sull'adozione di tecniche avanzate di bilanciamento per le classi minoritarie, passaggi fondamentali per migliorare ulteriormente la robustezza e l'accuratezza del sistema.

Riferimenti bibliografici

- [1] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>